

Documentação - Pipeline Automatizado de Extração e Carga de Dados

Este pipeline foi desenvolvido para automatizar a extração de relatórios de um sistema web, realizar o tratamento dos dados com Pandas e carregá-los em um Data Warehouse PostgreSQL com integridade referencial. O processo ainda inclui notificações via Telegram para monitoramento automático.

Visão Geral

- **Linguagem**: Python 3.12
- **Fonte de dados**: Portal Web (ex: TOTVS Analytics)
- **Destino**: Data Warehouse PostgreSQL
- **Monitoramento**: API do Telegram

Estrutura do Pipeline

1. Automação de Navegador (Selenium)

Arquivo/função: `baixar_relatorio_web()`

Responsável por:

- Iniciar o Chrome com perfil personalizado e diretório de download pré-definido
- Realizar login na plataforma de relatórios
- Navegar até o relatório desejado
- Exportar e baixar o relatório no formato `.xlsx`

Configurações importantes:

- `user-data-dir`: garante que sessões anteriores de login sejam reutilizadas
- `WebDriverWait`: aguarda carregamento de elementos antes de clicar ou interagir

2. ****Processamento de Dados (Pandas)****

****Arquivo/função****: `processar\_excel()`

Responsável por:

- Leitura do arquivo Excel, com remoção de linhas inválidas como "Rollup"
- Conversão de colunas para os tipos corretos
- Renomeação e padronização de colunas
- Tratamento de valores ausentes

Requisitos da estrutura:

- Deve haver uma coluna de data (` DATA `) e uma de valor (` VALOR `)

3. ****Conexão com PostgreSQL (psycopg2)****

****Arquivo principal****

- A conexão com o banco é feita diretamente via `psycopg2.connect()`
- Todas as interações com o banco são realizadas por cursores SQL manuais

4. ****Pré-processamento do Banco****

****Função****: `limpar\_tabelas()`

- Truncamento completo das tabelas fato e dimensões envolvidas
- ` CASCADE ` garante remoção de dependências
- Pode ser adaptado para suportar cargas parciais/incrementais

5. ****Inserção de Dados****

****Função****: `inserir\_dados()`

- Para cada linha do DataFrame:
- Verifica e insere, se necessário, os valores nas dimensões (tempo, entidade)
- Insere na tabela fato com as chaves estrangeiras

Práticas aplicadas:

- Verificações prévias com `SELECT` para evitar duplicações
- Inserção via `INSERT INTO ... VALUES (...);`
- `conn.commit()` apenas uma vez ao final para performance

6. ****Notificação via Telegram****

****Função**:** `enviar\_mensagem\_telegram()`

- Usa a API pública do Telegram para enviar mensagens para o chat definido
- Envia mensagens em caso de sucesso ou falha, com mensagens customizáveis

Modelo de Dados

Tabela: `dim\_tempo`

```sql

CREATE TABLE dim\_tempo (

id SERIAL PRIMARY KEY,

data DATE UNIQUE

);

```

Tabela: `dim\_entidade`

```sql

```
CREATE TABLE dim_entidade (
 id SERIAL PRIMARY KEY,
 nome TEXT UNIQUE
);
` ``
```

Tabela: `fato`

```
` `` `sql
CREATE TABLE fato (
 id SERIAL PRIMARY KEY,
 id_entidade INT REFERENCES dim_entidade(id),
 id_tempo INT REFERENCES dim_tempo(id),
 valor NUMERIC
);
` ``
```

---

## Configuração Inicial

1. Instale as dependências:
2. Configure o arquivo `.env` (recomendado):  
` `` `env  
TOKEN=seu\_token  
CHAT\_ID=seu\_chat\_id (id do bot)  
CAMINHO\_ARQUIVO=caminho/para/arquivo.xlsx  
` ``

3. Ajuste os XPaths e URLs no trecho Selenium para refletirem seu ambiente

4. Execute o script com:

```
```bash
```

```
python main.py
```

```
```
```

### **Resultados Esperados**

- Extração automatizada de relatórios Excel
- Dados tratados e normalizados
- Carga segura e referencial no PostgreSQL
- Notificação automática no Telegram

### **Possíveis Extensões**

- Agendamento com `Task Scheduler` ou `cron`
- Suporte a múltiplos relatórios
- Criação de painéis Power BI/Looker com base nas tabelas
- Adição de logs com `logging`
- Carga incremental com controle de datas

Para sugestões ou melhorias, contribua via PR no GitHub ou entre em contato.